



Universidad de San Carlos de Guatemala
-Centro Universitario de Occidente-
Ingeniería en Ciencias y Sistemas
Ing. Francisco Rojas
Auxiliar: Yefri González
Redes de Computadoras 2

Banco 5

Infraestructura de Red con Segmentación Interna Avanzada

PARTE INDIVIDUAL

Carnet	Nombre
201930672	Vicente Rocael Matías Osorio

Quetzaltenango septiembre de 2026

Índice

1. Introducción y objetivos
2. Arquitectura general de la red
3. Esquema de direccionamiento
4. Configuración del Router
5. Configuración del Switch (VLANs)
6. Firewall y control de tráfico
7. NAT
8. Servicios internos (Servidor 1 y Servidor 2)
9. Persistencia de configuración
10. Integración interbancaria (anillo de 5 bancos)
11. Enrutamiento redundante y failover
12. Seguridad y aislamiento entre bancos
13. Evidencias de Wireshark
14. HSRP y DMZ — justificación de no aplicabilidad
15. Cumplimiento del enunciado
16. Conclusiones
17. Anexos

1. Introducción y objetivos

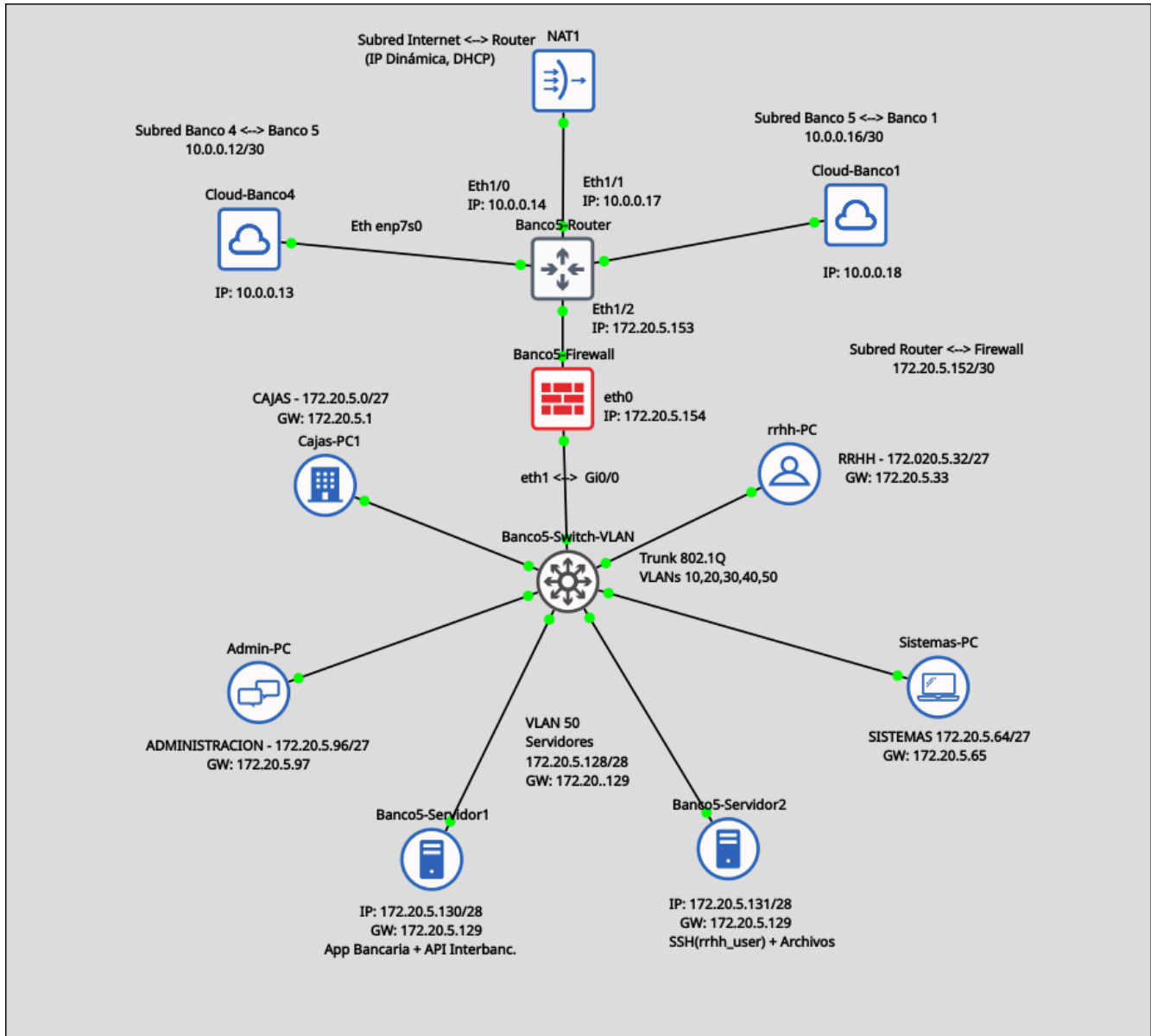
Este documento describe la implementación de Banco 5 dentro del proyecto grupal de Infraestructura de Red Bancaria, correspondiente al curso de Redes de Computadoras 2. El diseño de Banco 5 se basa en una segmentación interna avanzada: cinco VLANs con distintas políticas de acceso, un firewall central con estado (stateful) que controla todo el tráfico entre segmentos, NAT hacia Internet y hacia el anillo interbancario, enrutamiento 100% estático con rutas de respaldo, y monitoreo activo de disponibilidad (IP SLA + Object Tracking) para lograr failover automático sin intervención manual.

Además de los requisitos individuales, Banco 5 forma parte de un anillo de 5 bancos interconectados (Banco1 – Banco2 – Banco3 – Banco4 – Banco5 – Banco1), coordinados en tiempo real mediante un repositorio Git compartido entre los cinco estudiantes, lo que permitió probar en la práctica escenarios reales de caída de enlaces, transferencias interbancarias reales y recuperación automática de rutas.

Objetivos específicos:

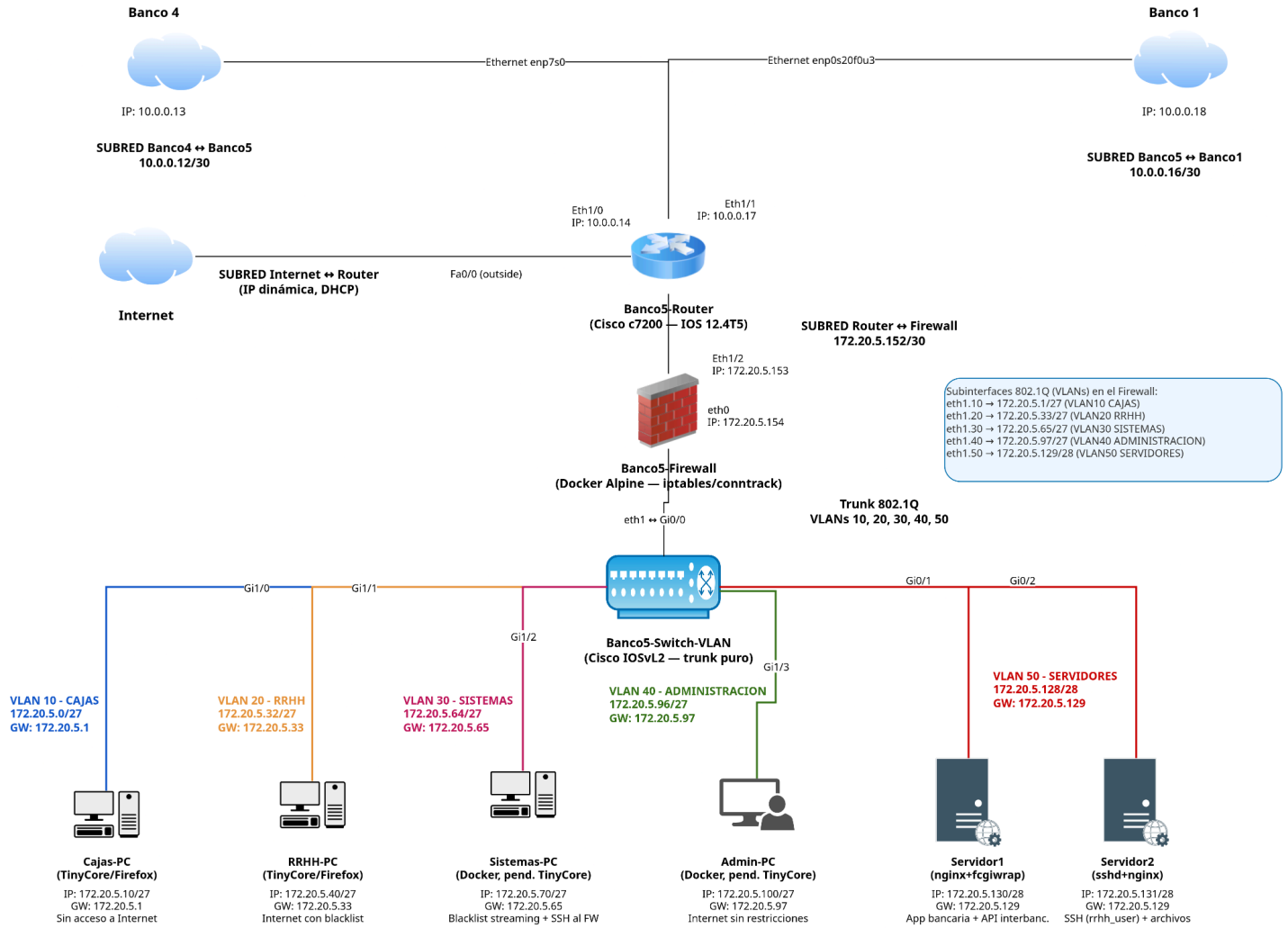
- Implementar una red completamente funcional en GNS3 usando exclusivamente enrutamiento estático.
- Segmentar la red interna en VLANs con políticas de firewall diferenciadas y verificadas.
- Implementar NAT de salida a Internet y NAT hacia el anillo interbancario.
- Exponer servicios de red reales (aplicación bancaria, transferencias interbancarias, SSH, visor de archivos).
- Garantizar continuidad de servicio ante la caída de un enlace del anillo, mediante rutas de respaldo activadas por monitoreo real (no solo por caída de interfaz local).
- Documentar y evidenciar cada uno de los puntos anteriores con capturas de tráfico reales.

2. Arquitectura general de la red



Topología de Red

Banco 5 -- Segmentación Interna Avanzada



Direccionamiento del anillo interbancario

Banco4 ↔ Banco5: 10.0.0.12/30 (B4=.13 / B5=.14)
Banco5 ↔ Banco1: 10.0.0.16/30 (B5=.17 / B1=.18)
Banco1 ↔ Banco2: 10.0.0.0/30 (.1 / .2)
Banco2 ↔ Banco3: 10.0.0.4/30 (.5 / .6)
Banco3 ↔ Banco4: 10.0.0.8/30 (.9 / .10)

Alta disponibilidad interbancaria

Primaria + respaldo en cada tramo del anillo:
10.0.0.4/30 y 10.0.0.8/30 → via Banco4 (track10, primaria) / via Banco1 (AD200, resp.)
10.0.0.0/30 → via Banco1 (track20, primaria) / via Banco4 (AD200, resp.)
IP SLA extremo-a-extremo (salto 2) anclado con PBR local
(RM-LOCAL-SLA) para no interferir con el tráfico real.

La arquitectura se organiza en tres capas: borde (router con NAT), distribución/filtrado (firewall central) y acceso (switch L2 + hosts/servidores). El switch es puramente de Capa 2 — todo el ruteo entre VLANs y todo el filtrado de tráfico se realizan en el firewall mediante subinterfaces 802.1Q e iptables con seguimiento de conexiones (conntrack).

Internet (Nodo Cloud, puenteado a wlan0 del host)

| DHCP

[Banco5-Router — Cisco c7200, IOS c7200-adventerprisek9-mz.124-24.T5]

| NAT overload (3 reglas independientes vía route-map)

| 172.20.5.153/30 (Ethernet1/2, inside)

[Banco5-Firewall — Docker Alpine]

| eth0 = 172.20.5.154/30 (hacia el router)

| eth1 = trunk 802.1Q (hacia el switch)

|— eth1.10 → 172.20.5.1/27 VLAN10 CAJAS

|— eth1.20 → 172.20.5.33/27 VLAN20 RRHH

|— eth1.30 → 172.20.5.65/27 VLAN30 SISTEMAS

|— eth1.40 → 172.20.5.97/27 VLAN40 ADMINISTRACION

|— eth1.50 → 172.20.5.129/28 VLAN50 SERVIDORES

[Banco5-Switch-VLAN — Cisco IOSvL2, trunk puro, sin routing L3]

|— Gi0/0 (trunk 802.1Q) → Firewall

|— Gi0/1 (access VLAN50) → Servidor1

|— Gi0/2 (access VLAN50) → Servidor2

|— Gi1/0 (access VLAN10) → Cajas-PC

|— Gi1/1 (access VLAN20) → RRHH-PC

|— Gi1/2 (access VLAN30) → Sistemas-PC

|— Gi1/3 (access VLAN40) → Admin-PC

Interconexión interbancaria (anillo físico):

[Banco5-Router] Ethernet1/0 (10.0.0.14/30) — Banco4 (10.0.0.13)

[Banco5-Router] Ethernet1/1 (10.0.0.17/30) — Banco1 (10.0.0.18)

Decisiones de diseño relevantes:

- El switch no tiene SVIs ni ip routing: se descartó el diseño inicial con ACLs directamente en el switch (stateless) porque exigía reglas separadas para el tráfico de retorno de los servidores. Centralizar el filtrado en el firewall resuelve esto automáticamente con ESTABLISHED,RELATED.
- El router solo hace NAT y borde de red — no filtra tráfico entre VLANs, esa responsabilidad es exclusiva del firewall.
- Todo el enrutamiento es estático, sin protocolos dinámicos, según lo exigido por el enunciado.

3. Esquema de direccionamiento

Segmento	Red	Gateway / IP
VLAN 10 — Cajas	172.20.5.0/27	172.20.5.1
VLAN 20 — RRHH	172.20.5.32/27	172.20.5.33
VLAN 30 — Sistemas	172.20.5.64/27	172.20.5.65
VLAN 40 — Administración	172.20.5.96/27	172.20.5.97
VLAN 50 — Servidores	172.20.5.128/28	172.20.5.129
Enlace Router↔Firewall	172.20.5.152/30	Router .153, Firewall .154
Interconexión → Banco4	10.0.0.12/30	Banco5 .14, Banco4 .13
Interconexión → Banco1	10.0.0.16/30	Banco5 .17, Banco1 .18

Nodo	IP	Rol
Cajas-PC	172.20.5.10/27	Cliente (TinyCore + Firefox)
RRHH-PC	172.20.5.40/27	Cliente (TinyCore + Firefox), con SSH hacia Servidor2
Sistemas-PC	172.20.5.70/27	Cliente (Docker Alpine)
Admin-PC	172.20.5.100/27	Cliente (Docker Alpine)
Servidor1	172.20.5.130/28	Aplicación bancaria + transferencias interbancarias
Servidor2	172.20.5.131/28	SSH + visor de archivos

4. Configuración del Router

Router Cisco c7200 con módulo PA-4E (slot 1), 5 interfaces disponibles (FastEthernet0/0 + Ethernet1/0 a 1/3). Configuración base de interfaces:

```
hostname Banco5-Router

interface Ethernet1/2
ip address 172.20.5.153 255.255.255.252
ip nat inside
no shutdown

interface Ethernet0/0
ip address dhcp
ip nat outside
no shutdown

interface Ethernet1/0
ip address 10.0.0.14 255.255.255.252
ip nat outside
no shutdown

interface Ethernet1/1
ip address 10.0.0.17 255.255.255.252
ip nat outside
no shutdown

ip route 172.20.5.0 255.255.255.0 172.20.5.154
ip route 0.0.0.0 0.0.0.0 dhcp
```

Banco5 está directamente conectado a Banco4 y a Banco1 (esos dos tramos del anillo no necesitan ruta estática, son redes conectadas). Los otros tres tramos del anillo (Banco1↔Banco2, Banco2↔Banco3, Banco3↔Banco4) sí la necesitan para tener visibilidad completa del anillo, cada uno con una ruta primaria por el camino corto y una de respaldo dando la vuelta completa, con distancia administrativa mayor para que solo se active si la primaria desaparece:

```
ip route 10.0.0.0 255.255.255.252 10.0.0.18 ! primaria via Banco1
ip route 10.0.0.0 255.255.255.252 10.0.0.13 200 ! respaldo via Banco4
ip route 10.0.0.4 255.255.255.252 10.0.0.13 ! primaria via Banco4
ip route 10.0.0.4 255.255.255.252 10.0.0.18 200 ! respaldo via Banco1
```



```
ip route 10.0.0.8 255.255.255.252 10.0.0.13 ! primaria via Banco4
ip route 10.0.0.8 255.255.255.252 10.0.0.18 200 ! respaldo via Banco1
```

El detalle completo de IP SLA, Object Tracking y PBR se documenta en la sección 11 (Enrutamiento redundante y failover), ya que es un tema transversal que evolucionó de forma significativa durante el desarrollo.

5. Configuración del Switch (VLANs)

```
hostname Banco5-Switch-VLAN
```

```
vlan 10 name CAJAS
```

```
vlan 20 name RRHH
```

```
vlan 30 name SISTEMAS
```

```
vlan 40 name ADMINISTRACION
```

```
vlan 50 name SERVIDORES
```

```
no ip routing
```

```
interface GigabitEthernet0/0
```

```
switchport trunk encapsulation dot1q
```

```
switchport mode trunk
```

```
switchport trunk allowed vlan 10,20,30,40,50
```

```
no shutdown
```

```
! Puertos de acceso (Gi0/1, Gi0/2, Gi1/0-1/3), ejemplo:
```

```
switchport mode access
```

```
switchport access vlan <N>
```

```
switchport port-security
```

```
switchport port-security maximum 1
```

```
switchport port-security mac-address sticky
```

```
switchport port-security violation restrict
```

```
spanning-tree portfast
```

```
no shutdown
```

Banco5-Switch-VLAN>show vlan brief

VLAN	Name	Status	Ports
1	default	active	Gi0/3, Gi2/0, Gi2/1, Gi2/2 Gi2/3, Gi3/0, Gi3/1, Gi3/2 Gi3/3
10	CAJAS	active	Gi1/0
20	RRHH	active	Gi1/1
30	SISTEMAS	active	Gi1/2
40	ADMINISTRACION	active	Gi1/3
50	SERVIDORES	active	Gi0/1, Gi0/2
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

Banco5-Switch-VLAN>

=== show vlan brief ===

VLAN Name	Status	Ports
1 default	active	Gi0/3, Gi2/0, Gi2/1, Gi2/2 Gi2/3, Gi3/0, Gi3/1, Gi3/2 Gi3/3
10 CAJAS	active	Gi1/0
20 RRHH	active	Gi1/1
30 SISTEMAS	active	Gi1/2
40 ADMINISTRACION	active	Gi1/3
50 SERVIDORES	active	Gi0/1, Gi0/2

=== Prueba de violacion real ===

Se cambio temporalmente la MAC de Sistemas-PC (puerto Gi1/2, VLAN30) de 02:42:8d:29:8a:00 (aprendida como SecureSticky) a 02:99:99:99:99:99, y se genero trafico real (ping hacia el gateway 172.20.5.65) desde la MAC nueva.

Resultado: 100% packet loss (el switch descarta el trafico de una MAC no autorizada en modo restrict, sin necesitar apagar el puerto).

=== show port-security interface GigabitEthernet1/2 ===

Port Security : Enabled
Port Status : Secure-up
Violation Mode : Restrict
Maximum MAC Addresses : 1
Total MAC Addresses : 1
Sticky MAC Addresses : 1
Last Source Address:Vlan : 0299.9999.9999:30
Security Violation Count : 12

=== show logging | include SECURITY ===

*Sep 10 09:30:19.313: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred, caused by MAC address 0299.9999.9999 on port GigabitEthernet1/2.
*Sep 10 09:30:32.030: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred, caused by MAC address 0299.9999.9999 on port GigabitEthernet1/2.
*Sep 10 09:30:42.464: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred, caused by MAC address 0299.9999.9999 on port GigabitEthernet1/2.
*Sep 10 09:30:47.798: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred, caused by MAC address 0299.9999.9999 on port GigabitEthernet1/2.

Tras la prueba, se restauro la MAC original de Sistemas-PC (02:42:8d:29:8a:00) y el puerto volvio a su estado normal sin necesitar "default interface"

Port Security: se probó en la práctica desconectando/reconectando un host y forzando un cambio de MAC — el puerto quedó bloqueado (modo restrict) hasta resetear con default interface, confirmando que la política sticky funciona correctamente.

6. Firewall y control de tráfico

El firewall corre en un contenedor Docker Alpine con 5 subinterfaces 802.1Q (una por VLAN) más eth0 hacia el router. Usa iptables con seguimiento de estado (conntrack) e ipset para bloqueo de dominios sin proxy.

6.1 Control de dominios por VLAN (sin proxy)

- **RRHH:** bloqueados facebook.com, instagram.com, tiktok.com, twitter.com
- **Sistemas:** bloqueados netflix.com, youtube.com, twitch.tv
- **Administración:** sin blacklist (acceso general)
- **Cajas:** sin acceso a Internet en absoluto
-

El bloqueo se implementa resolviendo cada dominio con getent ahostsv4 y poblando un ipset tipo hash:ip por VLAN, evitando así un proxy web (no permitido por el enunciado). El script apply_blacklist.sh se puede re-ejecutar en caliente para actualizar las IPs si el dominio cambia de resolución.

6.2 Reglas de firewall

```
iptables -P INPUT DROP
```

```
iptables -P FORWARD DROP
```

```
iptables -P OUTPUT ACCEPT
```

```
iptables -A INPUT -i lo -j ACCEPT
```

```
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

```
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

```
iptables -A INPUT -i eth1.30 -p tcp --dport 22 -j ACCEPT
```

! Cajas -> Servidor1 (app bancaria, HTTP)

```
iptables -A FORWARD -i eth1.10 -o eth1.50 -d 172.20.5.130 -p tcp --dport 80 -j ACCEPT
```

! RRHH -> Servidor2 (SSH + visor web) y control de blacklist

```
iptables -A FORWARD -i eth1.20 -o eth1.50 -d 172.20.5.131 -p tcp --dport 22 -j ACCEPT
```

```
iptables -A FORWARD -i eth1.20 -o eth1.50 -d 172.20.5.131 -p tcp --dport 80 -j ACCEPT
```

```
iptables -A FORWARD -i eth1.20 -o eth0 -m set --match-set blacklist_rrhh dst -j DROP
```

```
iptables -A FORWARD -i eth1.20 -o eth0 -j ACCEPT
```

! Sistemas -> Servidor1/Servidor2 y control de blacklist

```
iptables -A FORWARD -i eth1.30 -o eth1.50 -d 172.20.5.130 -p tcp -m multiport --dports 22,80 -j ACCEPT
```

```
iptables -A FORWARD -i eth1.30 -o eth1.50 -d 172.20.5.131 -p tcp --dport 22 -j ACCEPT
```

```
iptables -A FORWARD -i eth1.30 -o eth0 -m set --match-set blacklist_sistemas dst -j DROP
```

```
iptables -A FORWARD -i eth1.30 -o eth0 -j ACCEPT
```

! Administración -> Servidor1 y acceso general

```
iptables -A FORWARD -i eth1.40 -o eth1.50 -d 172.20.5.130 -p tcp --dport 80 -j ACCEPT
```

```
iptables -A FORWARD -i eth1.40 -o eth0 -j ACCEPT
```

! Excepciones puntuales: Servidor1 -> APIs interbancarias (Banco3 y Banco4)

```
iptables -A FORWARD -i eth1.50 -o eth0 -d 10.0.0.13 -p tcp --dport 8080 -j ACCEPT
```

```
iptables -A FORWARD -i eth1.50 -o eth0 -d 10.0.0.10 -p tcp --dport 8080 -j ACCEPT
```

```
iptables -A FORWARD -i eth1.50 -o eth0 -d 10.0.0.6 -p tcp --dport 80 -j ACCEPT
```

```
iptables -A FORWARD -i eth1.50 -o eth0 -d 10.0.0.9 -p tcp --dport 80 -j ACCEPT
```

! Evidencia: registrar (sin bloquear más de lo que ya bloqueaba el DROP por defecto)

! cualquier intento del anillo hacia nuestros servidores internos

```
iptables -A FORWARD -i eth0 -m limit --limit 10/min -j LOG \
```

```
--log-prefix "FWD-BLOCKED-FROM-RING: " --log-level 4
```

VLAN	Internet	Blacklist propia	Acceso a Servidor1	Acceso a Servidor2
Cajas	No	—	HTTP (app bancaria)	No
RRHH	Sí	Redes sociales	No	SSH + HTTP
Sistemas	Sí	Streaming	SSH + HTTP	SSH
Administración	Sí	Ninguna	HTTP	No
Servidores	No general	—	—	—

Nota de diseño: la VLAN de Servidores no tiene una regla general de salida a Internet — solo las excepciones puntuales hacia las IPs y puertos específicos de las APIs de Banco3/Banco4. Esto es mínimo privilegio real: un servidor comprometido no puede "llamar a casa" a cualquier destino arbitrario de Internet.

7. NAT

Se implementan tres reglas de NAT overload independientes, una por interfaz de salida (Internet, Banco4, Banco1), todas sobre el mismo rango de origen interno (172.20.5.0/24):

```
ip access-list standard NAT_ACL_INTERNET
 permit 172.20.5.0 0.0.0.255
ip access-list standard NAT_ACL_BANCO4
 permit 172.20.5.0 0.0.0.255
ip access-list standard NAT_ACL_BANCO1
 permit 172.20.5.0 0.0.0.255
```

```
route-map NAT_RM_INTERNET permit 10
 match ip address NAT_ACL_INTERNET
 match interface FastEthernet0/0
route-map NAT_RM_BANCO4 permit 10
 match ip address NAT_ACL_BANCO4
 match interface Ethernet1/0
route-map NAT_RM_BANCO1 permit 10
 match ip address NAT_ACL_BANCO1
 match interface Ethernet1/1
```

```
ip nat inside source route-map NAT_RM_INTERNET interface FastEthernet0/0 overload
ip nat inside source route-map NAT_RM_BANCO4 interface Ethernet1/0 overload
ip nat inside source route-map NAT_RM_BANCO1 interface Ethernet1/1 overload
```

Decisión de equipo (anillo de 5 bancos): ocultar las IPs privadas internas de cada banco — cada banco solo ve la IP de borde del router vecino, nunca la subred interna real. Esto también evita que los bancos vecinos necesiten rutas hacia 172.20.5.0/24.

7.1 Evidencia de traducción NAT

Captura del tráfico **antes** de traducir (en eth0 del Firewall, origen real sin traducir):

```
23:53:00.027356 IP 172.20.5.130.56306 > 10.0.0.13.8080: Flags [P.], seq 1:100 ... HTTP: GET /blacklist
HTTP/1.1
```

Tabla de traducción activa en el router, capturada mientras la misma conexión seguía viva (show ip nat translations):

Pro	Inside global	Inside local	Outside local	Outside global
tcp	10.0.0.14:50420	172.20.5.130:50420	10.0.0.13:8080	10.0.0.13:8080

Se observa la traducción real: 172.20.5.130:56306/50420 (IP interna de Servidor1) se convierte en 10.0.0.14:* (IP de borde de Banco5) antes de salir hacia Banco4 — exactamente el comportamiento esperado de NAT overload (PAT).

8. Servicios internos

8.1 Servidor1 — Aplicación bancaria

Corre nginx + fcgiwrap, sirviendo una aplicación CGI en bash con:

- index.cgi: tabla de cuentas + formulario de transferencia local + formulario de transferencia interbancaria a Banco3 + consulta de blacklist de Banco4.
- transferir.cgi: transferencias locales, valida fondos y actualiza el archivo plano cuentas.txt (formato id|nombre|saldo).
- transferencia_banco3.cgi: transferencias salientes reales hacia la API de Banco3 (ver sección 10).
- blacklist_banco4.cgi: consulta la API real de Banco4 (GET /blacklist) y muestra el resultado.
- api/consulta.cgi: endpoint tipo API para que otros bancos consulten cuentas (parte grupal).

Banco 5 - Aplicacion Banca... x

172.20.5.130/cgi-bin/index.cgi

Banco 5

Servicio de Aplicacion Bancaria

Cuentas

No. Cuenta	Titular	Saldo
1001	Cajero Principal	Q 5000.00
1002	Cliente A	Q 1500.00
1003	Cliente B	Q 800.00

Firefox automatically sends some data to Mozilla so that we can improve your experience. Choose What I Share x

Banco 5 - Aplicacion Banca... x

172.20.5.130/cgi-bin/index.cgi Google

Realizar Transferencia

Cuenta origen:

Cuenta destino:

Monto:

Transferir

Firefox automatically sends some data to Mozilla so that we can improve your experience. Choose What I Share x

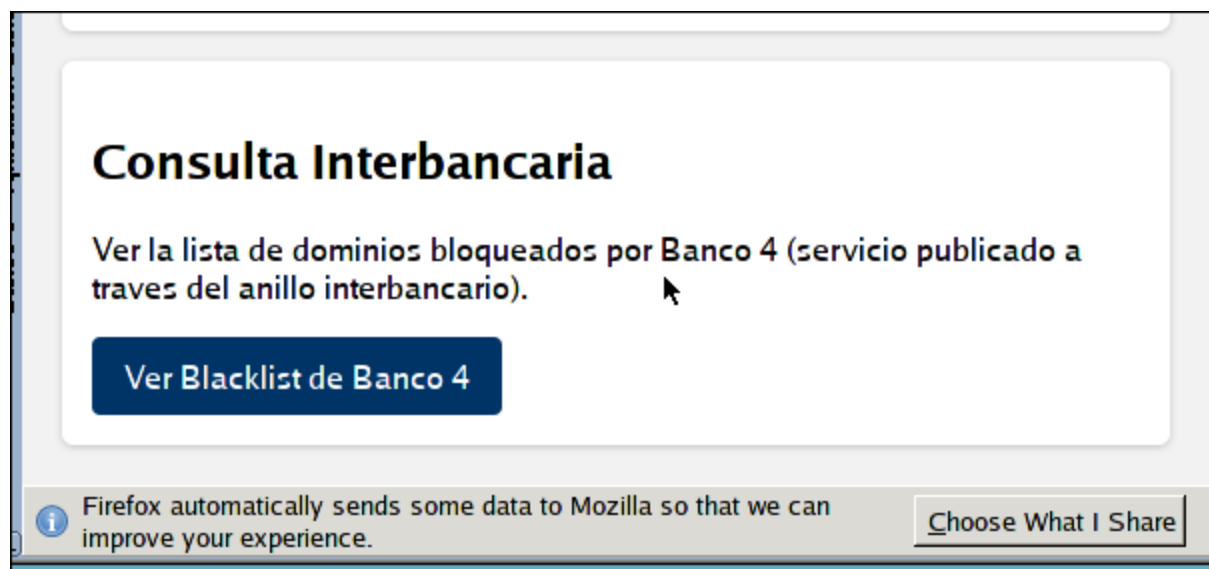
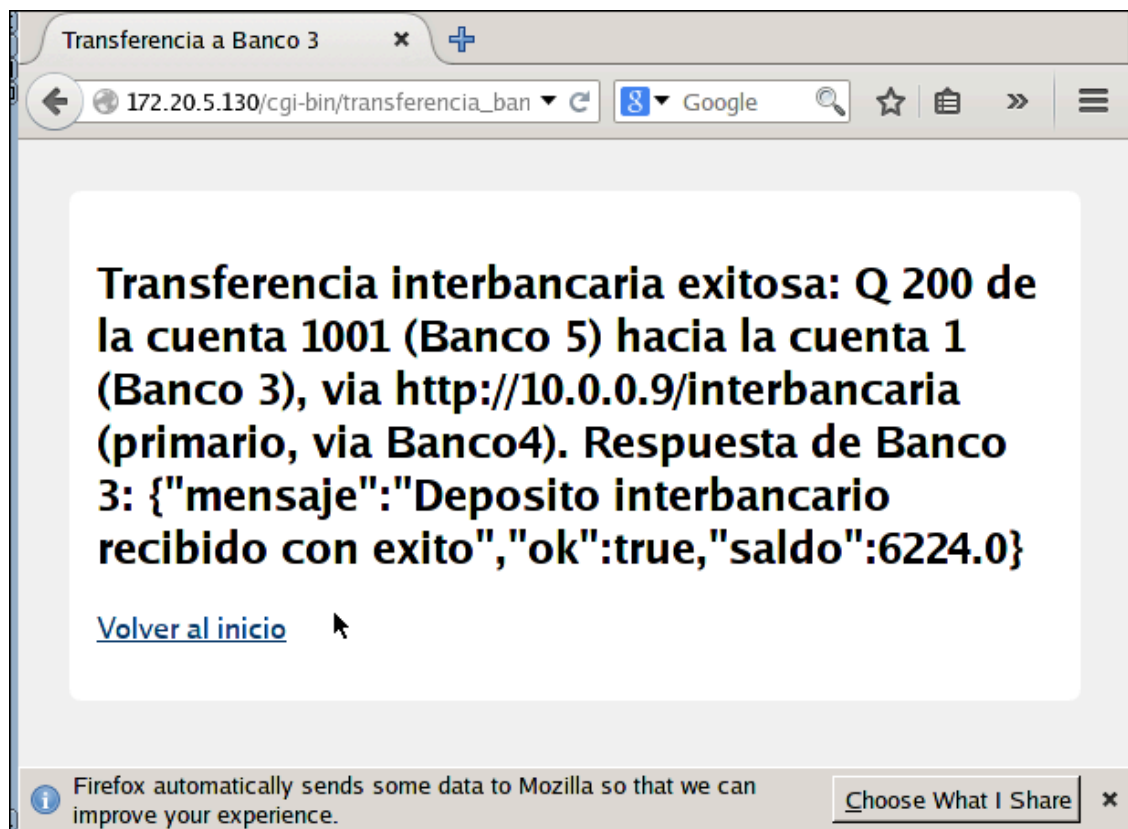
Resultado x

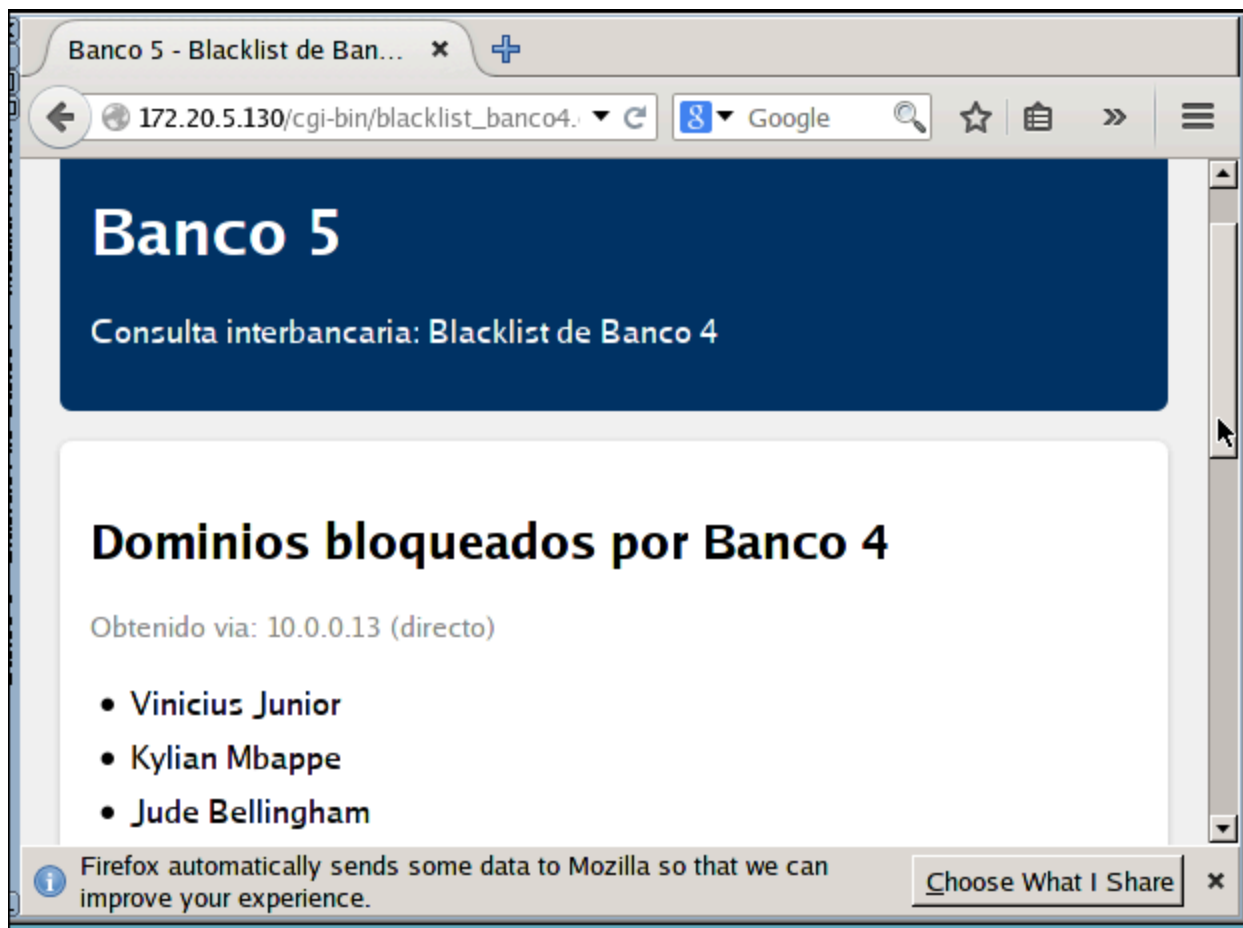
172.20.5.130/cgi-bin/transferir.cgi Google

Transferencia realizada: Q 200 de 1001 hacia 1002.

[Volver al inicio](#)

Firefox automatically sends some data to Mozilla so that we can improve your experience. Choose What I Share x





8.2 Servidor2 — SSH y visor de archivos

Expone SSH (usuario rrhh_user, autenticación por contraseña) y un visor de archivos estático servido por nginx (autoindex off: solo se puede acceder a un archivo si se conoce el enlace directo, no se puede listar el directorio).

9. Persistencia de configuración

scripts/

— firewall.sh	# interfaces, blacklist e iptables del Firewall
— Srv1.sh	# app bancaria + transferencias + consulta a Banco4
— Srv2.sh	# SSH + visor de archivos
— deploy_scripts.sh	# script maestro: localiza los contenedores por nombre # y distribuye + ejecuta el script correspondiente

Uso: `./scripts/deploy_scripts.sh` (los tres nodos) o `./scripts/deploy_scripts.sh firewall|srv1|srv2` (uno solo). Los tres scripts son idempotentes: se pueden re-ejecutar sin recrear el contenedor, sin duplicar reglas ni dejar procesos huérfanos. Se probó exitosamente después de varios reinicios reales de la computadora y del proyecto de GNS3 durante el desarrollo.

Bug corregido durante el desarrollo: en estos contenedores el PID 1 es un `/bin/sh` simple que nunca reaporta procesos hijos; si un demonio (`nginx/sshd`) muere queda "zombie" y `rc-service X restart` falla. Los scripts evitan esto matando el proceso viejo con `kill -9` y arrancando el binario directo, en vez de depender de OpenRC.

10. Integración interbancaria (anillo de 5 bancos)

Banco5 participa en un anillo de 5 bancos coordinados en tiempo real mediante un repositorio Git compartido entre los cinco estudiantes, usado para sincronizar el estado de la red, diagnosticar problemas cruzados y coordinar pruebas de failover reales sin necesitar estar todos conectados a la misma llamada.

10.1 Transferencias interbancarias salientes hacia Banco3

Cajas inicia la transferencia desde el portal de Servidor1 (mismo patrón que la consulta a Banco4: el cliente nunca sale directo hacia el anillo, es Servidor1 quien hace la llamada del lado del servidor — no es un proxy web genérico, es composición de datos con un endpoint fijo y conocido).

Protocolo acordado con Banco3:

POST http://10.0.0.9/interbancaria

Content-Type: application/json

Body: {"cuenta_origen": <id>, "cuenta_destino": <id>, "monto": <numero>}

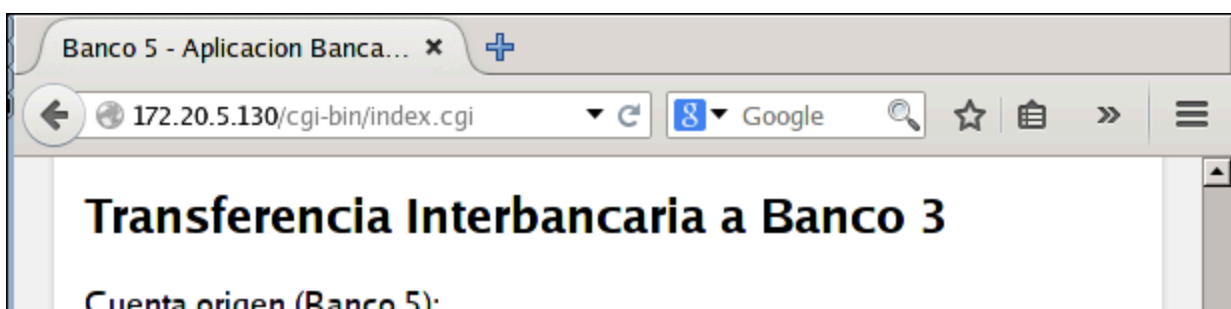
Éxito -> 200 OK con {"ok": true, "mensaje": "...", "saldo": <nuevo_saldo>}

Banco5 valida fondos y **debita localmente antes** de llamar a la API. Si la respuesta es 200 OK con "ok":true, se confirma (commit); si falla o da timeout, se revierte el débito local (rollback). Banco3 nunca toca cuentas de Banco5 directamente.

Resiliencia ante el failover: la ruta a Banco3 puede salir por dos caminos distintos, y Banco3 empareja la IP de origen con la IP de destino esperada (según cuál de sus propios enlaces recibe el tráfico). El script prueba primero el endpoint correspondiente a la ruta primaria (vía Banco4, destino 10.0.0.9) con un timeout corto, y si no responde "ok":true, reintenta automáticamente con el endpoint de la ruta de respaldo (vía Banco1, destino 10.0.0.6) antes de decidir un rollback.

Prueba real exitosa (transferencia end-to-end): cuenta 1001 (Banco5) → cuenta 1 (Banco3), Q1.00. Saldo antes Q4400.00, saldo después Q4399.00. Respuesta real de Banco3:

```
{"mensaje":"Deposito interbancario recibido con exito","ok":true,"saldo":1811.0}
```



10.2 Consulta interbancaria a Banco4 (blacklist)

Servidor1 consulta en vivo la API pública de Banco4 (GET <http://10.0.0.13:8080/blacklist>) y renderiza el resultado. Igual que con las transferencias, si la ruta primaria hacia 10.0.0.13 no

responde, reintenta automáticamente contra 10.0.0.10:8080 (la otra interfaz de Banco4, alcanzable por la ruta de respaldo vía Banco1) — la página muestra explícitamente cuál de las dos rutas respondió, útil como evidencia de failover funcionando en la práctica.

11. Enrutamiento redundante y failover

11.1 Rutas flotantes (primer nivel)

Cada uno de los tres tramos ajenos del anillo tiene una ruta primaria (distancia administrativa 1) y una de respaldo dando la vuelta completa (distancia administrativa 200). Al tener mayor distancia, la ruta de respaldo solo se instala en la tabla de enrutamiento si la primaria desaparece del RIB — el mecanismo básico de Cisco IOS para failover estático.

Limitación de este primer nivel: una ruta flotante normal solo reacciona si la interfaz local se cae, no si el vecino directo sigue arriba pero él mismo perdió su propio siguiente salto.

11.2 IP SLA + Object Tracking (segundo nivel — salto 1)

Se agregó monitoreo activo con sondas ICMP cada 5 segundos hacia el vecino directamente conectado, con un temporizador de estabilización (delay down 10 up 5) para evitar falsos positivos por un solo ping perdido:

```
ip sla 1
  icmp-echo 10.0.0.13 source-interface Ethernet1/0
  frequency 5
ip sla schedule 1 life forever start-time now

track 1 ip sla 1 reachability
  delay down 10 up 5

ip route 10.0.0.4 255.255.255.252 10.0.0.13 track 1
```

Bug real encontrado y corregido: el diseño original apuntaba las sondas directamente al destino final (ej. 10.0.0.6), que vive dentro de la misma red que la propia ruta con track controla. En cuanto el track bajaba y la ruta pasaba a respaldo, la siguiente sonda automáticamente empezaba a salir por el camino de respaldo — si ese respaldo también fallaba, el track se quedaba atascado en *Down* permanentemente, incluso después de que el camino primario se

hubiera recuperado. Se corrigió apuntando las sondas al *next-hop directamente conectado* (10.0.0.13, 10.0.0.18) en vez del destino final, ya que esas IPs viven en redes conectadas que ningún track controla.

11.3 Monitoreo de extremo a extremo — salto 2 (PBR local)

A partir de una guía compartida por Banco4 en el repositorio de coordinación, se implementó monitoreo de "vecino del vecino": en un anillo de 5 nodos, 2 saltos en cada dirección dan visibilidad del 100% de la topología — es decir, detectar no solo si Banco4 está vivo, sino si Banco4 puede llegar más allá, hasta Banco3.

Riesgo identificado en la plantilla original (no se copió tal cual): proponía anclar cada sonda con una ruta /32 fija sin track. Como una ruta /32 siempre gana por longest-prefix-match, esa ruta fija habría anclado también el tráfico *real* de las transferencias interbancarias hacia esa misma IP, rompiendo el failover del servicio bancario a cambio de ganar visibilidad de red.

Solución adoptada — política de ruteo local (ip local policy route-map): en vez de una ruta que afecta todo el tráfico hacia esa IP, el route-map solo actúa sobre paquetes *originados por el propio router* (las sondas IP SLA); el tráfico real de tránsito sigue la tabla de rutas normal, con su track y su respaldo intactos.

```
ip access-list extended ACL-SLA-B4-B3
permit icmp host 10.0.0.14 host 10.0.0.9
route-map RM-LOCAL-SLA permit 10
match ip address ACL-SLA-B4-B3
set ip next-hop 10.0.0.13
ip local policy route-map RM-LOCAL-SLA

ip sla 10
icmp-echo 10.0.0.9 source-interface Ethernet1/0
frequency 5
ip sla schedule 10 life forever start-time now
track 10 ip sla 10 reachability
delay down 10 up 5

ip route 10.0.0.4 255.255.255.252 10.0.0.13 track 10
ip route 10.0.0.8 255.255.255.252 10.0.0.13 track 10
```

Esta misma técnica ya la usa Banco3 en su propio router (confirmado directamente con su equipo), lo que valida el enfoque como consistente con el resto del anillo.

11.4 Pruebas de failover realizadas en vivo

Prueba	Resultado
Apagar Ethernet1/0 (comando shutdown)	Ruta de respaldo activa al instante, [200/0] via 10.0.0.18
Desconectar el cable físico real (enp7s0 a NO-CARRIER)	Tracks caen en 10-90s, rutas conmutan a respaldo automáticamente
ping 10.0.0.9 con el enlace primario caído	100% de éxito (5/5), RTT ~52-64ms, por la ruta larga
traceroute 10.0.0.9 con el enlace primario caído	Camino completo: 10.0.0.18 (Banco1) → 10.0.0.2 (Banco2) → 10.0.0.6 (Banco3)
Reconectar el enlace primario	Tracks vuelven a Up, rutas regresan solas a la primaria, sin intervención manual

f

Distinción importante: si Banco4 sigue vivo (responde ping al 100%) pero él mismo no puede reenviar hacia Banco3, el failover de Banco5 *no se activa a propósito* — el diseño de salto 1 solo monitorea "¿el vecino directo está vivo?"; para eso se agregó el monitoreo de salto 2 (sección 11.3), que sí detecta esa condición y monitorea de forma segura sin afectar el tráfico real.

12. Seguridad y aislamiento entre bancos

Se verificó explícitamente que ningún banco vecino del anillo puede alcanzar los servicios internos de Banco5 (aplicación bancaria, archivos, SSH) sin pasar por una regla de excepción explícita:

- La política por defecto de la cadena FORWARD es DROP.
- Las únicas excepciones hacia eth0 (frontera con el anillo) son las 4 reglas puntuales de

las APIs interbancarias (sección 6.2), todas con destino y puerto específicos — nunca una regla general de entrada desde el anillo hacia las VLANs internas.

- Se agregó una regla de solo registro (LOG, no terminante) para dejar evidencia de cualquier intento de tráfico desde el anillo que no coincida con esas excepciones — no cambia el comportamiento de bloqueo, solo lo documenta.

Prueba realizada: ping desde el router hacia Servidor1 (172.20.5.130) simulando tráfico entrante desde la frontera del anillo — **0% de éxito**, y el contador de la regla de registro incrementó exactamente en la cantidad de paquetes enviados, confirmando que el firewall efectivamente bloquea y además deja constancia del intento (ver evidencia en sección 13.5).

13. Evidencias de Wireshark

Todas las capturas se realizaron con tcpdump ejecutado directamente dentro del contenedor del Firewall (genera archivos .pcap estándar, 100% compatibles con Wireshark) en el punto de la red donde converge todo el tráfico relevante — la frontera entre VLANs y la frontera hacia el anillo interbancario. Los 6 archivos .pcap/evidencia están en la carpeta capturas/ del repositorio del proyecto.

13.1 ARP

Punto de captura: eth1.10 del Firewall. Se limpió la caché de vecinos y se generó tráfico real hacia Cajas-PC.

```
02:42:4d:13:be:01 → Broadcast ARP Who has 172.20.5.10? Tell 172.20.5.1
0c:eb:7b:90:00:00 → 02:42:4d:13:be:01 ARP 172.20.5.10 is at 0c:eb:7b:90:00:00
172.20.5.1 → 172.20.5.10 ICMP Echo (ping) request
172.20.5.10 → 172.20.5.1 ICMP Echo (ping) reply
```

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

Aplique un filtro de visualización ... <Ctrl-/>

No.	Time	Delta	Source	Destination	Protocol	Length	Info
1	0.000000		0c:1f:b5:cc:00:00	PVST+	STP	64	Conf. Root = 32768/10/0c:1f:b5:cc:00:00 Cost = 0 Port = 0x8001
2	0.057064	0.057064	02:42:4d:13:be:01	Broadcast	ARP	42	Who has 172.20.5.10? Tell 172.20.5.1
3	0.060275	0.003211	0c:eb:7b:90:00:00	02:42:4d:13:be:01	ARP	60	172.20.5.10 is at 0c:eb:7b:90:00:00
4	0.060297	0.000022	172.20.5.1	172.20.5.10	ICMP	98	Echo (ping) request id=0xedc, seq=1/256, ttl=64 (reply in 5)
5	0.062490	0.002193	172.20.5.10	172.20.5.1	ICMP	98	Echo (ping) reply id=0xedc, seq=1/256, ttl=64 (request in 4)
6	1.058242	0.995752	172.20.5.1	172.20.5.10	ICMP	98	Echo (ping) request id=0xedc, seq=2/512, ttl=64 (reply in 7)
7	1.061607	0.003365	172.20.5.10	172.20.5.1	ICMP	98	Echo (ping) reply id=0xedc, seq=2/512, ttl=64 (request in 6)
8	2.017424	0.955817	0c:1f:b5:cc:00:00	PVST+	STP	64	Conf. Root = 32768/10/0c:1f:b5:cc:00:00 Cost = 0 Port = 0x8001
9	2.058999	0.041575	172.20.5.1	172.20.5.10	ICMP	98	Echo (ping) request id=0xedc, seq=3/768, ttl=64 (reply in 10)
10	2.061613	0.002614	172.20.5.10	172.20.5.1	ICMP	98	Echo (ping) reply id=0xedc, seq=3/768, ttl=64 (request in 9)

Frame 1: Packet, 64 bytes on wire (512 bits), 64 bytes captured (512 bits)

IEEE 802.3 Ethernet

Logical-Link Control

Spanning Tree Protocol

0000 01 00 0c cc cc cd 0c 1f b5 cc 00 00 00 32 aa aa2..
0010 03 00 00 0c 01 0b 00 00 00 00 00 80 0a 0c 1f b5
0020 cc 00 00 00 00 00 00 80 0a 0c 1f b5 cc 00 00 80
0030 01 00 00 14 00 02 00 0f 00 00 00 00 02 00 0a2..

13.2 ICMP

Punto de captura: eth1.50 del Firewall (Firewall ↔ Servidor1), 4 pares completos de solicitud/respuesta:

172.20.5.129 → 172.20.5.130 ICMP Echo (ping) request seq=1

172.20.5.130 → 172.20.5.129 ICMP Echo (ping) reply seq=1

172.20.5.129 → 172.20.5.130 ICMP Echo (ping) request seq=2

172.20.5.130 → 172.20.5.129 ICMP Echo (ping) reply seq=2

... (4 pares en total, 100% de respuesta)

No.	Time	Delta	Source	Destination	Protocol	Length	Info
1	0.000000		172.20.5.129	172.20.5.130	ICMP	98	Echo (ping) request id=0x0edd, seq=1/256, ttl=64 (reply in 2)
2	0.001199	0.001199	172.20.5.130	172.20.5.129	ICMP	98	Echo (ping) reply id=0x0edd, seq=1/256, ttl=64 (request in 1)
3	1.000785	0.999586	172.20.5.129	172.20.5.130	ICMP	98	Echo (ping) request id=0x0edd, seq=2/512, ttl=64 (reply in 4)
4	1.003419	0.002634	172.20.5.130	172.20.5.129	ICMP	98	Echo (ping) reply id=0x0edd, seq=2/512, ttl=64 (request in 3)
5	2.001768	0.998349	172.20.5.129	172.20.5.130	ICMP	98	Echo (ping) request id=0x0edd, seq=3/768, ttl=64 (reply in 6)
6	2.003783	0.002015	172.20.5.130	172.20.5.129	ICMP	98	Echo (ping) reply id=0x0edd, seq=3/768, ttl=64 (request in 5)
7	3.002947	0.999164	172.20.5.129	172.20.5.130	ICMP	98	Echo (ping) request id=0x0edd, seq=4/1024, ttl=64 (reply in 8)
8	3.009807	0.006860	172.20.5.130	172.20.5.129	ICMP	98	Echo (ping) reply id=0x0edd, seq=4/1024, ttl=64 (request in 7)

Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)	0000	02 42 ed 06 8f 00 02 42	4d 13 be 01 08 00 45 00	B B M E
Ethernet II, Src: 02:42:4d:13:be:01 (02:42:4d:13:be:01), Dst: 02:42:ed:06:8f:00 (02:42:ed:06:8f:00)	0010	00 54 dc a6 40 00 40 01	fa d6 ac 14 05 81 ac 14	T . @ @
Internet Protocol Version 4, Src: 172.20.5.129, Dst: 172.20.5.130	0020	05 82 08 00 25 c5 0e dd	00 01 0e 29 a2 6a 00 00	. . . % j . .
Internet Control Message Protocol	0030	00 00 53 f6 00 00 00 00	00 00 10 11 12 13 14 15	. S
	0040	16 17 18 19 1a 1b 1c 1d	1e 1f 20 21 22 23 24 25	 I * \$ %
	0050	26 27 28 29 2a 2b 2c 2d	2e 2f 30 31 32 33 34 35	8 ' () * + , - . / 0 1 2 3 4 5
	0060	36 37		6 7

13.3 TCP / HTTP

Punto de captura: eth0 del Firewall. Handshake TCP completo, petición HTTP real y cierre de la conexión, generado por blacklist_banco4.cgi consultando la API real de Banco4:

172.20.5.130 → 10.0.0.13 TCP [SYN]

10.0.0.13 → 172.20.5.130 TCP [SYN, ACK]

172.20.5.130 → 10.0.0.13 TCP [ACK]

172.20.5.130 → 10.0.0.13 HTTP GET /blacklist HTTP/1.1

10.0.0.13 → 172.20.5.130 HTTP HTTP/1.1 200 OK

172.20.5.130 → 10.0.0.13 TCP [FIN, ACK]

No.	Time	Delta	Source	Destination	Protocol	Length	Info
1	0.000000		172.20.5.130	10.0.0.13	TCP	74	45770 → 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2018555154 TSecr=0 WS=1024
2	0.038720	0.038720	10.0.0.13	172.20.5.130	TCP	74	8080 → 45770 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=3896427327 TSecr=2018555154
3	0.041820	0.003100	172.20.5.130	10.0.0.13	TCP	66	45770 → 8080 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2018555196 TSecr=3896427327
4	0.042056	0.000236	172.20.5.130	10.0.0.13	HTTP	165	GET /blacklist HTTP/1.1
5	0.069062	0.027006	10.0.0.13	172.20.5.130	TCP	66	8080 → 45770 [ACK] Seq=1 Ack=100 Win=65152 Len=0 TSval=3896427357 TSecr=2018555196
6	0.069129	0.000067	10.0.0.13	172.20.5.130	TCP	466	8080 → 45770 [PSH, ACK] Seq=1 Ack=100 Win=65152 Len=400 TSval=3896427359 TSecr=2018555196 [TCP PD...]
7	0.076745	0.007616	172.20.5.130	10.0.0.13	TCP	66	45770 → 8080 [ACK] Seq=100 Ack=401 Win=64512 Len=0 TSval=2018555228 TSecr=3896427359
8	0.079172	0.002427	10.0.0.13	172.20.5.130	HTTP	312	HTTP/1.1 200 OK (text/plain)

10.0.0.13 → 172.20.5.130 TCP [ACK]

13.4 UDP / DNS

Punto de captura: eth0 del Firewall. 7 consultas DNS reales (tipo A) disparadas por apply_blacklist.sh al resolver los dominios bloqueados, hacia el servidor 8.8.8.8:

172.20.5.154 → 8.8.8.8 DNS Standard query A facebook.com

8.8.8.8 → 172.20.5.154 DNS Standard query response A facebook.com A 57.144.22.1

172.20.5.154 → 8.8.8.8 DNS Standard query A instagram.com

8.8.8.8 → 172.20.5.154 DNS Standard query response A instagram.com A 57.144.22.34

... (7 dominios consultados en total: facebook, instagram, tiktok, twitter, netflix, youtube, twitch)

No.	Time	Delta	Source	Destination	Protocol	Length	Info
1	0.000000		172.20.5.154	8.8.8.8	DNS	72	Standard query 0x382f A facebook.com
2	0.115296	0.115296	8.8.8.8	172.20.5.154	DNS	88	Standard query response 0x382f A facebook.com A 57.144.22.1
3	0.119192	0.003896	172.20.5.154	8.8.8.8	DNS	73	Standard query 0xf252 A instagram.com
4	0.236123	0.116931	8.8.8.8	172.20.5.154	DNS	89	Standard query response 0xf252 A instagram.com A 57.144.22.34
5	0.242570	0.006447	172.20.5.154	8.8.8.8	DNS	70	Standard query 0x4f9e A tiktok.com
6	0.346949	0.104379	8.8.8.8	172.20.5.154	DNS	102	Standard query response 0x4f9e A tiktok.com A 23.50.115.196 A 23.50.115.219
7	0.355794	0.008845	172.20.5.154	8.8.8.8	DNS	71	Standard query 0xe46b A twitter.com
8	0.427473	0.071679	8.8.8.8	172.20.5.154	DNS	87	Standard query response 0xe46b A twitter.com A 162.159.140.229
9	0.433823	0.006350	172.20.5.154	8.8.8.8	DNS	71	Standard query 0x9f61 A netflix.com
10	0.518161	0.084338	8.8.8.8	172.20.5.154	DNS	119	Standard query response 0x9f61 A netflix.com A 54.160.93.182 A 3.211.157.115 A 3.225.92.8
11	0.529675	0.011514	172.20.5.154	8.8.8.8	DNS	71	Standard query 0x2ca7 A youtube.com
12	0.618833	0.089158	8.8.8.8	172.20.5.154	DNS	87	Standard query response 0x2ca7 A youtube.com A 142.251.215.14
13	0.625085	0.006252	172.20.5.154	8.8.8.8	DNS	69	Standard query 0x8b5e A twitch.tv
14	0.719753	0.094668	8.8.8.8	172.20.5.154	DNS	133	Standard query response 0x8b5e A twitch.tv A 151.101.66.167 A 151.101.194.167 A 151.101.130.167 A...

▶ Frame 1: Packet, 72 bytes on wire (576 bits), 72 bytes captured (576 bits)	0000	ca 01 aa 69 00 1e 02 42	4d 13 be 00 08 00 45 00	...i...B M...E:
▶ Ethernet II, Src: 02:42:4d:13:be:00 (02:42:4d:13:be:00), Dst: ca:01:aa:69:00:1e (ca:01:aa:69:00:1e)	0010	00 3a 06 9b 40 00 40 11	72 5a ac 14 05 9a 08 08	...: @ @ rZ.....
▶ Internet Protocol Version 4, Src: 172.20.5.154, Dst: 8.8.8.8	0020	08 08 e1 6d 00 35 00 26	a9 00 38 2f 01 00 00 01	...m 5 6 ..8/....
▶ User Datagram Protocol, Src Port: 57709, Dst Port: 53	0030	00 00 00 00 00 00 08 66	61 63 65 62 6f 6f 6b 03	f acebook
▶ Domain Name System (query)	0040	63 6f 6d 00 00 01 00 01		com.....

13.5 Tráfico bloqueado

Punto de captura: eth0 del Firewall. 5 solicitudes ICMP entrando desde el router (simulando tráfico desde la frontera del anillo) hacia 172.20.5.130, **sin ninguna respuesta**:

```
172.20.5.153 → 172.20.5.130 ICMP Echo (ping) request seq=0
172.20.5.153 → 172.20.5.130 ICMP Echo (ping) request seq=1
172.20.5.153 → 172.20.5.130 ICMP Echo (ping) request seq=2
172.20.5.153 → 172.20.5.130 ICMP Echo (ping) request seq=3
172.20.5.153 → 172.20.5.130 ICMP Echo (ping) request seq=4
(0 respuestas — bloqueado por la política DROP por defecto)
```

Confirmado desde el router: ping 172.20.5.130 → **0% de éxito (0/5)**. En paralelo, el contador de la regla de registro (sección 12) pasó de 0 a exactamente 5 paquetes (500 bytes), confirmando que el firewall vio y registró cada intento antes de descartarlo:

```
pkts bytes target prot opt in out source destination
 5  500 LOG    all  -- eth0 *  0.0.0.0/0 0.0.0.0/0 limit: avg 10/min burst 5
                                LOG flags 0 level 4 prefix "FWD-BLOCKED-FROM-RING: "
```

14. HSRP y DMZ — justificación de no aplicabilidad

HSRP: no aplica en el diseño de Banco5 porque no existen gateways redundantes — cada VLAN tiene un único firewall como gateway, y la redundancia del diseño se resuelve a nivel de anillo (rutas primarias/respaldo con IP SLA), no a nivel de gateway local. Agregar HSRP hubiera significado duplicar el firewall completo (con todo su estado de conntrack e ipset), lo cual añade complejidad sin un beneficio real dado que el punto único de falla real del diseño no es el gateway sino la conectividad hacia el anillo, que ya está cubierta.

DMZ: no aplica en la parte individual porque el perfil de Banco5 no expone servicios

directamente a Internet — los únicos servicios expuestos (aplicación bancaria, transferencias, SSH, archivos) son consumidos desde dentro de la propia red interna o desde el anillo interbancario (con NAT y reglas de firewall específicas), nunca desde Internet público. Un DMZ tendría sentido si se planeara publicar un servicio bancario hacia Internet, lo cual está fuera del alcance definido para esta parte del proyecto.

15. Cumplimiento del enunciado

Requisito	Estado
GNS3 exclusivamente, routing estático	Cumplido
VLAN con políticas reales probadas	Cumplido (5 VLANs, matriz de firewall)
NAT de salida (Internet)	Cumplido, confirmado con traducciones reales
NAT hacia el anillo (route-map)	Cumplido, confirmado con Banco4
Firewall con mínimo privilegio	Cumplido
Control de dominios sin proxy	Cumplido (ipset + iptables por VLAN)
Mínimo 2 servicios de red	Superado (app bancaria + transferencias + API interbancaria; SSH + visor de archivos)
Port Security	Cumplido, con evidencia de violaciones
HSRP	No aplica (justificado en sección 14)
DMZ	No aplica (justificado en sección 14)
Wireshark (ARP, ICMP, TCP, UDP, NAT, permitido/bloqueado)	Cumplido — 6 evidencias (sección 13)
Persistencia de configuración tras recreación de contenedores	Resuelto con scripts de restauración

Requisito	Estado
Transferencias interbancarias salientes a Banco3	Completa y verificada con transferencia real
Rutas de respaldo (anillo completo)	Configuradas y verificadas con pruebas reales de desconexión

16. Conclusiones

- El diseño centralizado de filtrado en el firewall (en vez de ACLs distribuidas en el switch) simplificó enormemente el manejo del tráfico de retorno gracias al seguimiento de estado, y resultó ser la decisión más importante de toda la arquitectura.
- El enrutamiento estático con rutas flotantes es suficiente para una topología pequeña y estable, pero por sí solo no detecta fallas más allá del vecino directo — el monitoreo activo con IP SLA y Object Tracking, primero a un salto y luego a dos, fue necesario para lograr un failover realmente confiable de punta a punta.
- Trabajar dentro de un anillo real de 5 bancos, coordinado con otros compañeros en tiempo real, expuso problemas que nunca hubieran aparecido probando Banco5 de forma aislada — bugs de NAT, condiciones de carrera en el failover, y discrepancias de documentación entre bancos que solo se resuelven con pruebas cruzadas reales.
- La persistencia de configuración en contenedores Docker fue un problema recurrente y no trivial durante todo el desarrollo; resolverlo con scripts versionados en vez de una imagen horneada permitió seguir iterando la configuración sin perder el histórico de decisiones de diseño.

17. Anexos

Los scripts completos de restauración (firewall.sh, Srv1.sh, Srv2.sh, deploy_scripts.sh), el script de monitoreo de failover, diagrama de topología y todas las capturas de Wireshark.